

SQL SELECT Mastery — Complete Syllabus

0. Course Philosophy

The student should progress through this sequence:

SELECT
↓
SELECT + WHERE
↓
SELECT + Expressions
↓
SELECT + ORDER BY
↓
SELECT + DISTINCT
↓
SELECT + Aggregate Functions
↓
SELECT + GROUP BY
↓
SELECT + HAVING
↓
SELECT + CASE
↓
SELECT + JOIN
↓
SELECT + Multiple JOINS
↓
SELECT + Subqueries
↓
SELECT + EXISTS / NOT EXISTS
↓
SELECT + UNION / INTERSECT / EXCEPT
↓
SELECT + CTE
↓
SELECT + Window Functions
↓
SELECT + Advanced Analytics

PART 1 — SELECT FUNDAMENTALS

Module 1 — Understanding SELECT

Topics

- What is **SELECT**?
- Why SELECT is the heart of SQL
- Basic SELECT syntax
- Selecting columns
- Selecting multiple columns
- Selecting all columns
- Column aliases
- Expressions
- Constants
- Calculated columns

Basic syntax

```
SELECT column1, column2  
FROM table_name;
```

Examples

```
SELECT product_id  
FROM products;  
SELECT product_id, product_name, price  
FROM products;  
SELECT *  
FROM products;
```

Aliases

```
SELECT  
    product_name AS product,  
    price AS selling_price  
FROM products;
```

Calculated columns

```
SELECT
    product_name,
    price,
    quantity,
    price * quantity AS total_value
FROM products;
```

Module 2 — SELECT Expressions

Learn how SQL can calculate values while retrieving data.

Topics

- Arithmetic operators
- Addition
- Subtraction
- Multiplication
- Division
- Modulus
- String concatenation
- Date expressions
- Boolean expressions

Examples

```
SELECT
    product_name,
    price,
    price * 1.18 AS price_with_tax
FROM products;
```

```
SELECT
    product_name,
    price,
    price * 0.90 AS discounted_price
FROM products;
```

Module 3 — DISTINCT

Topics

- Removing duplicates
- DISTINCT on one column
- DISTINCT on multiple columns
- DISTINCT vs GROUP BY

```
SELECT DISTINCT category
FROM products;
SELECT DISTINCT category, brand
FROM products;
```

Important

```
SELECT DISTINCT category
FROM products;
```

returns unique categories.

But:

```
SELECT DISTINCT category, brand
FROM products;
```

returns unique **combinations** of category + brand.

Module 4 — WHERE

This is one of the most important SQL modules.

Topics

- Basic filtering
- Comparison operators
- =
- <>
- !=
- >
- <
- >=
- <=

Examples

```
SELECT *  
FROM products  
WHERE price > 1000;  
SELECT *  
FROM products  
WHERE category = 'Electronics';  
SELECT *  
FROM products  
WHERE price BETWEEN 500 AND 2000;
```

Module 5 — Logical Operators

Topics

- AND
- OR
- NOT
- Operator precedence
- Parentheses

```
SELECT *  
FROM products  
WHERE category = 'Electronics'  
AND price > 50000;  
SELECT *  
FROM products  
WHERE category = 'Electronics'  
OR category = 'Furniture';
```

Important

Understand:

NOT
AND
OR

and always use parentheses when the logic is complex.

```
SELECT *  
FROM products  
WHERE
```

```
(category = 'Electronics' AND price > 50000)
OR
(category = 'Furniture' AND price > 10000);
```

Module 6 — IN

```
SELECT *
FROM products
WHERE category IN
(
    'Electronics',
    'Furniture',
    'Clothing'
);
```

NOT IN

```
SELECT *
FROM products
WHERE category NOT IN
(
    'Electronics',
    'Furniture'
);
```

Module 7 — BETWEEN

```
SELECT *
FROM products
WHERE price BETWEEN 1000 AND 5000;
```

Important:

BETWEEN is inclusive.

Conceptually:

```
price >= 1000
AND price <= 5000
```

Module 8 — LIKE

Topics

- %
- _
- Prefix search
- Suffix search
- Contains search
- Pattern matching

```
SELECT *  
FROM customers  
WHERE customer_name LIKE 'A%';
```

Starts with A.

```
SELECT *  
FROM customers  
WHERE customer_name LIKE '%a';
```

Ends with a.

```
SELECT *  
FROM customers  
WHERE customer_name LIKE '%an%';
```

Contains an.

```
SELECT *  
FROM customers  
WHERE customer_name LIKE '_a%';
```

Second character is a.

Module 9 — NULL

Extremely important.

Topics

- What is NULL?
- NULL vs 0
- NULL vs empty string
- IS NULL
- IS NOT NULL
- NULL comparison
- NULL propagation

Correct:

```
SELECT *  
FROM customers  
WHERE phone IS NULL;
```

Incorrect:

```
WHERE phone = NULL;
```

Correct:

```
SELECT *  
FROM customers  
WHERE phone IS NOT NULL;
```

Module 10 — ORDER BY

Topics

- Ascending
- Descending
- Multiple columns
- Alias ordering
- Expression ordering

```
SELECT  
    product_name,  
    price  
FROM products  
ORDER BY price;  
SELECT  
    product_name,  
    price  
FROM products
```


ORDER BY price DESC;

Multiple columns:

```
SELECT *  
FROM products  
ORDER BY category, price DESC;
```

Module 11 — LIMIT / TOP / FETCH

Different databases use different syntax.

PostgreSQL / MySQL

```
SELECT *  
FROM products  
ORDER BY price DESC  
LIMIT 10;
```

SQL Server

```
SELECT TOP 10 *  
FROM products  
ORDER BY price DESC;
```

Standard-style pagination

```
SELECT *  
FROM products  
ORDER BY product_id  
OFFSET 20 ROWS  
FETCH NEXT 10 ROWS ONLY;
```

PART 2 — SQL FUNCTIONS

Module 12 — String Functions

Learn:

UPPER

LOWER
LENGTH
TRIM
LTRIM
RTRIM
SUBSTRING
CONCAT
REPLACE
LEFT
RIGHT

Example:

```
SELECT
    customer_name,
    UPPER(customer_name) AS uppercase_name,
    LOWER(customer_name) AS lowercase_name
FROM customers;
SELECT
    customer_name,
    LENGTH(customer_name) AS name_length
FROM customers;
```

Module 13 — Numeric Functions

Examples:

```
SELECT
    product_name,
    price,
    ROUND(price, 2) AS rounded_price
FROM products;
```

Also learn:

ROUND
CEIL / CEILING
FLOOR
ABS
MOD
POWER
SQRT

Module 14 — Date Functions

Topics:

- Current date
- Current timestamp
- Extracting year
- Extracting month
- Extracting day
- Date arithmetic
- Date difference
- Date filtering

Example:

```
SELECT
    order_id,
    order_date
FROM orders
WHERE order_date >= '2026-01-01';
```

Example:

```
SELECT
    order_id,
    EXTRACT(YEAR FROM order_date) AS order_year
FROM orders;
```

Module 15 — Aggregate Functions

Master:

```
COUNT
SUM
AVG
MIN
MAX
```

COUNT

```
SELECT COUNT(*) AS total_orders  
FROM orders;
```

SUM

```
SELECT SUM(order_amount) AS revenue  
FROM orders;
```

AVG

```
SELECT AVG(price) AS average_price  
FROM products;
```

MIN / MAX

```
SELECT  
    MIN(price) AS minimum_price,  
    MAX(price) AS maximum_price  
FROM products;
```

PART 3 — GROUPING

Module 16 — GROUP BY

```
SELECT  
    category,  
    COUNT(*) AS product_count  
FROM products  
GROUP BY category;
```

Sales:

```
SELECT  
    customer_id,  
    SUM(order_amount) AS total_spent  
FROM orders  
GROUP BY customer_id;
```

Module 17 — GROUP BY Multiple Columns

```
SELECT
    category,
    brand,
    COUNT(*) AS product_count
FROM products
GROUP BY category, brand;
```

Module 18 — HAVING

Critical distinction:

WHERE → filters rows
HAVING → filters groups

Example:

```
SELECT
    customer_id,
    SUM(order_amount) AS total_spent
FROM orders
GROUP BY customer_id
HAVING SUM(order_amount) > 100000;
```

Module 19 — WHERE + GROUP BY + HAVING

```
SELECT
    category,
    AVG(price) AS average_price
FROM products
WHERE price > 1000
GROUP BY category
HAVING AVG(price) > 5000
ORDER BY average_price DESC;
```

This is an important milestone.

PART 4 — CONDITIONAL SELECT

Module 20 — CASE

One of the most powerful SELECT features.

```
SELECT
  product_name,
  price,
  CASE
    WHEN price >= 100000 THEN 'Premium'
    WHEN price >= 50000 THEN 'Mid Range'
    ELSE 'Budget'
  END AS price_category
FROM products;
```

Module 21 — CASE + Aggregation

```
SELECT
  SUM(
    CASE
      WHEN order_status = 'Completed'
      THEN order_amount
      ELSE 0
    END
  ) AS completed_revenue
FROM orders;
```

Module 22 — COALESCE

```
SELECT
  customer_name,
  COALESCE(phone, 'Not Available') AS phone
```

FROM customers;

Also learn:

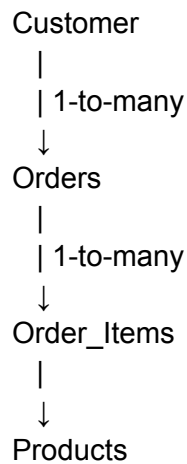
NULLIF
COALESCE
CASE

PART 5 — JOINS

This should be a major part of the course.

Module 23 — Understanding Relationships

Understand:



Module 24 — INNER JOIN

```
SELECT
  c.customer_name,
  o.order_id,
  o.order_date,
  o.order_amount
FROM customers c
INNER JOIN orders o
```

ON c.customer_id = o.customer_id;

Module 25 — LEFT JOIN

```
SELECT
  c.customer_name,
  o.order_id
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id;
```

Find customers without orders:

```
SELECT
  c.customer_id,
  c.customer_name
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Module 26 — RIGHT JOIN

Understand the concept and portability.

```
SELECT *
FROM customers c
RIGHT JOIN orders o
  ON c.customer_id = o.customer_id;
```

Module 27 — FULL OUTER JOIN

```
SELECT *
FROM customers c
FULL OUTER JOIN orders o
  ON c.customer_id = o.customer_id;
```

Module 28 — CROSS JOIN

```
SELECT
  c.customer_name,
  p.product_name
FROM customers c
CROSS JOIN products p;
```

Understand the Cartesian product.

Module 29 — SELF JOIN

Example employee hierarchy:

```
SELECT
  e.employee_name,
  m.employee_name AS manager_name
FROM employees e
LEFT JOIN employees m
  ON e.manager_id = m.employee_id;
```

Module 30 — Multiple JOINS

```
SELECT
  c.customer_name,
  o.order_id,
  p.product_name,
  oi.quantity,
  oi.unit_price
FROM customers c
JOIN orders o
  ON c.customer_id = o.customer_id
JOIN order_items oi
  ON o.order_id = oi.order_id
JOIN products p
  ON oi.product_id = p.product_id;
```

This is where students begin writing real-world SQL.

PART 6 — SUBQUERIES

Module 31 — Scalar Subquery

Find products above average price:

```
SELECT
    product_name,
    price
FROM products
WHERE price >
(
    SELECT AVG(price)
    FROM products
);
```

Module 32 — Subquery with IN

```
SELECT *
FROM customers
WHERE customer_id IN
(
    SELECT customer_id
    FROM orders
    WHERE order_amount > 100000
);
```

Module 33 — Correlated Subquery

```
SELECT
    p.product_id,
    p.product_name,
    p.price
FROM products p
WHERE p.price >
```

```
(  
  SELECT AVG(p2.price)  
  FROM products p2  
  WHERE p2.category = p.category  
);
```

Meaning:

Find products whose price is greater than the average price of their own category.

Module 34 — EXISTS

```
SELECT  
  c.customer_id,  
  c.customer_name  
FROM customers c  
WHERE EXISTS  
(  
  SELECT 1  
  FROM orders o  
  WHERE o.customer_id = c.customer_id  
);
```

Module 35 — NOT EXISTS

```
SELECT  
  c.customer_id,  
  c.customer_name  
FROM customers c  
WHERE NOT EXISTS  
(  
  SELECT 1  
  FROM orders o  
  WHERE o.customer_id = c.customer_id  
);
```

PART 7 — SET OPERATORS

Module 36 — UNION

```
SELECT customer_name AS name  
FROM customers
```

UNION

```
SELECT employee_name AS name  
FROM employees;
```

Removes duplicates.

Module 37 — UNION ALL

```
SELECT customer_name AS name  
FROM customers
```

UNION ALL

```
SELECT employee_name AS name  
FROM employees;
```

Keeps duplicates.

Module 38 — INTERSECT

```
SELECT customer_id  
FROM orders_2025
```

INTERSECT

```
SELECT customer_id  
FROM orders_2026;
```

Find customers present in both.

Module 39 — EXCEPT

```
SELECT customer_id  
FROM customers
```

```
EXCEPT
```

```
SELECT customer_id  
FROM orders;
```

Find customers who have never ordered.

PART 8 — CTE

Module 40 — Common Table Expressions

```
WITH customer_sales AS  
(  
    SELECT  
        customer_id,  
        SUM(order_amount) AS total_sales  
    FROM orders  
    GROUP BY customer_id  
)  
SELECT *  
FROM customer_sales  
WHERE total_sales > 100000;
```

Module 41 — Multiple CTEs

```
WITH customer_sales AS  
(  
    SELECT  
        customer_id,  
        SUM(order_amount) AS total_sales  
    FROM orders  
    GROUP BY customer_id
```

```

),
customer_orders AS
(
    SELECT
        customer_id,
        COUNT(*) AS order_count
    FROM orders
    GROUP BY customer_id
)
SELECT
    cs.customer_id,
    cs.total_sales,
    co.order_count
FROM customer_sales cs
JOIN customer_orders co
    ON cs.customer_id = co.customer_id;

```

Module 42 — Recursive CTE

Advanced SELECT topic.

Example organizational hierarchy:

WITH RECURSIVE employee_tree AS

```

(
    SELECT
        employee_id,
        employee_name,
        manager_id,
        1 AS level
    FROM employees
    WHERE manager_id IS NULL

```

UNION ALL

```

SELECT
    e.employee_id,
    e.employee_name,
    e.manager_id,
    et.level + 1
FROM employees e
JOIN employee_tree et

```

```
        ON e.manager_id = et.employee_id
    )
    SELECT *
    FROM employee_tree;
```

PART 9 — WINDOW FUNCTIONS

This should be the advanced portion of the course.

Module 43 — Window Function Fundamentals

Understand:

OVER()

Example:

```
SELECT
    employee_name,
    salary,
    AVG(salary) OVER() AS company_average_salary
FROM employees;
```

Module 44 — PARTITION BY

```
SELECT
    employee_name,
    department,
    salary,
    AVG(salary) OVER(
        PARTITION BY department
    ) AS department_avg_salary
FROM employees;
```

Module 45 — ROW_NUMBER

```
SELECT
    employee_name,
    department,
    salary,
    ROW_NUMBER() OVER(
        PARTITION BY department
        ORDER BY salary DESC
    ) AS rn
FROM employees;
```

Module 46 — RANK

```
SELECT
    employee_name,
    salary,
    RANK() OVER(
        ORDER BY salary DESC
    ) AS salary_rank
FROM employees;
```

Module 47 — DENSE_RANK

```
SELECT
    employee_name,
    salary,
    DENSE_RANK() OVER(
        ORDER BY salary DESC
    ) AS salary_rank
FROM employees;
```

Understand:

```
ROW_NUMBER
RANK
DENSE_RANK
```

Module 48 — LAG

```
SELECT
    order_date,
    order_amount,
    LAG(order_amount) OVER(
        ORDER BY order_date
    ) AS previous_order_amount
FROM orders;
```

Module 49 — LEAD

```
SELECT
    order_date,
    order_amount,
    LEAD(order_amount) OVER(
        ORDER BY order_date
    ) AS next_order_amount
FROM orders;
```

Module 50 — Running Total

```
SELECT
    order_date,
    order_amount,
    SUM(order_amount) OVER(
        ORDER BY order_date
    ) AS running_total
FROM orders;
```

Module 51 — Moving Average

```
SELECT
    order_date,
    order_amount,
    AVG(order_amount) OVER(
        ORDER BY order_date
```

```
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) AS seven_day_average
FROM orders;
```

Module 52 — Top N per Group

One of the most important interview problems.

```
WITH ranked_products AS
(
    SELECT
        product_name,
        category,
        price,
        ROW_NUMBER() OVER(
            PARTITION BY category
            ORDER BY price DESC
        ) AS rn
    FROM products
)
SELECT *
FROM ranked_products
WHERE rn <= 3;
```

PART 10 — SELECT CLAUSE ORDER

Students must understand this extremely well.

Although we write:

```
SELECT
FROM
WHERE
GROUP BY
HAVING
ORDER BY
```

SQL logically processes approximately:

```
FROM
```

↓
JOIN
↓
WHERE
↓
GROUP BY
↓
HAVING
↓
SELECT
↓
DISTINCT
↓
ORDER BY
↓
LIMIT

This explains many SQL errors.

PART 11 — FULL APPLICATION

Application: Retail Sales Analytics System

Build one complete application around:

Business

ABC Retail

The company sells:

Electronics
Furniture
Clothing
Books
Home Appliances
Sports

Database Model

Use these logical tables:

customers

customer_id
customer_name
email
city
state
country
registration_date

products

product_id
product_name
category_id
brand
price
cost
stock_quantity

categories

category_id
category_name

orders

order_id
customer_id
order_date
order_status
sales_channel

order_items

order_item_id
order_id
product_id

quantity
unit_price
discount

employees

employee_id
employee_name
department_id
manager_id
salary

departments

department_id
department_name

Application Query Modules

Dashboard 1 — Business Overview

Total customers

```
SELECT COUNT(*) AS total_customers  
FROM customers;
```

Total products

```
SELECT COUNT(*) AS total_products  
FROM products;
```

Total orders

```
SELECT COUNT(*) AS total_orders  
FROM orders;
```

Total revenue

```
SELECT  
    SUM(quantity * unit_price * (1 - discount)) AS revenue  
FROM order_items;
```

Dashboard 2 — Product Analytics

Products above average price

```
SELECT
    product_id,
    product_name,
    price
FROM products
WHERE price >
(
    SELECT AVG(price)
    FROM products
)
ORDER BY price DESC;
```

Most expensive products

```
SELECT
    product_name,
    price
FROM products
ORDER BY price DESC
LIMIT 10;
```

Average price by category

```
SELECT
    c.category_name,
    AVG(p.price) AS average_price
FROM products p
JOIN categories c
    ON p.category_id = c.category_id
GROUP BY c.category_name
ORDER BY average_price DESC;
```

Dashboard 3 — Customer Analytics

Customers with orders

```
SELECT DISTINCT
  c.customer_id,
  c.customer_name
FROM customers c
JOIN orders o
  ON c.customer_id = o.customer_id;
```

Customers without orders

```
SELECT
  c.customer_id,
  c.customer_name
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Customer revenue

```
SELECT
  c.customer_id,
  c.customer_name,
  SUM(
    oi.quantity *
    oi.unit_price *
    (1 - oi.discount)
  ) AS total_revenue
FROM customers c
JOIN orders o
  ON c.customer_id = o.customer_id
JOIN order_items oi
  ON o.order_id = oi.order_id
GROUP BY
  c.customer_id,
  c.customer_name
ORDER BY total_revenue DESC;
```

Dashboard 4 — Sales Analytics

Sales by month

```
SELECT
  EXTRACT(YEAR FROM o.order_date) AS year,
```

```
EXTRACT(MONTH FROM o.order_date) AS month,  
SUM(  
    oi.quantity *  
    oi.unit_price *  
    (1 - oi.discount)  
) AS revenue  
FROM orders o  
JOIN order_items oi  
    ON o.order_id = oi.order_id  
GROUP BY  
    EXTRACT(YEAR FROM o.order_date),  
    EXTRACT(MONTH FROM o.order_date)  
ORDER BY year, month;
```

Dashboard 5 — Top Products

```
SELECT  
    p.product_id,  
    p.product_name,  
    SUM(oi.quantity) AS units_sold,  
    SUM(  
        oi.quantity *  
        oi.unit_price *  
        (1 - oi.discount)  
    ) AS revenue  
FROM products p  
JOIN order_items oi  
    ON p.product_id = oi.product_id  
GROUP BY  
    p.product_id,  
    p.product_name  
ORDER BY revenue DESC  
LIMIT 10;
```

Dashboard 6 — Category Performance

```
SELECT  
    c.category_name,  
    COUNT(DISTINCT o.order_id) AS orders,  
    SUM(oi.quantity) AS units_sold,
```



```
SUM(
    oi.quantity *
    oi.unit_price *
    (1 - oi.discount)
) AS revenue
FROM categories c
JOIN products p
    ON c.category_id = p.category_id
JOIN order_items oi
    ON p.product_id = oi.product_id
JOIN orders o
    ON oi.order_id = o.order_id
GROUP BY c.category_name
ORDER BY revenue DESC;
```

Dashboard 7 — Customer Segmentation

```
SELECT
    customer_id,
    total_spent,
    CASE
        WHEN total_spent >= 500000 THEN 'VIP'
        WHEN total_spent >= 200000 THEN 'Premium'
        WHEN total_spent >= 50000 THEN 'Regular'
        ELSE 'Low Value'
    END AS customer_segment
FROM
(
    SELECT
        o.customer_id,
        SUM(
            oi.quantity *
            oi.unit_price *
            (1 - oi.discount)
        ) AS total_spent
    FROM orders o
    JOIN order_items oi
        ON o.order_id = oi.order_id
    GROUP BY o.customer_id
) x;
```

Dashboard 8 — Ranking

```
SELECT
    customer_id,
    total_spent,
    RANK() OVER(
        ORDER BY total_spent DESC
    ) AS customer_rank
FROM
(
    SELECT
        o.customer_id,
        SUM(
            oi.quantity *
            oi.unit_price *
            (1 - oi.discount)
        ) AS total_spent
    FROM orders o
    JOIN order_items oi
        ON o.order_id = oi.order_id
    GROUP BY o.customer_id
) x;
```

Dashboard 9 — Top 3 Customers per State

```
WITH customer_sales AS
(
    SELECT
        c.state,
        c.customer_id,
        c.customer_name,
        SUM(
            oi.quantity *
            oi.unit_price *
            (1 - oi.discount)
        ) AS revenue
    FROM customers c
    JOIN orders o
        ON c.customer_id = o.customer_id
    JOIN order_items oi
        ON o.order_id = oi.order_id
    GROUP BY
```

```

        c.state,
        c.customer_id,
        c.customer_name
    ),
    ranked_customers AS
    (
        SELECT
            *,
            ROW_NUMBER() OVER(
                PARTITION BY state
                ORDER BY revenue DESC
            ) AS rn
        FROM customer_sales
    )
    SELECT *
    FROM ranked_customers
    WHERE rn <= 3;

```

Dashboard 10 — Running Revenue

```

WITH daily_sales AS
(
    SELECT
        o.order_date,
        SUM(
            oi.quantity *
            oi.unit_price *
            (1 - oi.discount)
        ) AS daily_revenue
    FROM orders o
    JOIN order_items oi
        ON o.order_id = oi.order_id
    GROUP BY o.order_date
)
SELECT
    order_date,
    daily_revenue,
    SUM(daily_revenue) OVER(
        ORDER BY order_date
    ) AS cumulative_revenue
FROM daily_sales
ORDER BY order_date;

```

SQL SELECT Important Things to Remember

Rule 1

Never use:

WHERE SUM(...)

Use:

HAVING SUM(...)

Rule 2

Never compare NULL using:

= NULL

Use:

IS NULL

Rule 3

WHERE works before grouping.

WHERE → rows

GROUP BY → groups

HAVING → groups

Rule 4

Be careful with LEFT JOIN.

This:

```
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
```

is different from:

```
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
WHERE o.order_status = 'Completed'
```

The second condition can effectively eliminate unmatched rows.

Rule 5

Understand duplicate multiplication.

If:

Customer → Orders → Order Items

one customer can produce many rows after joining.

Therefore:

COUNT(*)

may not be the same as:

COUNT(DISTINCT customer_id)

SELECT Interview Patterns

Students should master these patterns.

1. Above average

```
WHERE salary >
(
  SELECT AVG(salary)
```

```
FROM employees  
)
```

2. Second highest

```
SELECT MAX(salary)  
FROM employees  
WHERE salary <  
(  
    SELECT MAX(salary)  
    FROM employees  
);
```

3. Duplicate values

```
SELECT  
    email,  
    COUNT(*) AS cnt  
FROM customers  
GROUP BY email  
HAVING COUNT(*) > 1;
```

4. Highest salary per department

```
WITH ranked AS  
(  
    SELECT  
        *,  
        RANK() OVER(  
            PARTITION BY department_id  
            ORDER BY salary DESC  
        ) AS rnk  
    FROM employees  
)  
SELECT *  
FROM ranked  
WHERE rnk = 1;
```

5. Customers with no orders

```
SELECT c.*  
FROM customers c  
WHERE NOT EXISTS  
(  
    SELECT 1  
    FROM orders o  
    WHERE o.customer_id = c.customer_id
```

);

6. Top 3 products

```
SELECT *  
FROM products  
ORDER BY price DESC  
LIMIT 3;
```

7. Top 3 per category

```
WITH ranked AS  
(  
    SELECT  
        *,  
        ROW_NUMBER() OVER(  
            PARTITION BY category_id  
            ORDER BY price DESC  
        ) AS rn  
    FROM products  
)  
SELECT *  
FROM ranked  
WHERE rn <= 3;
```

QUIZ

Beginner

Q1

Which clause filters individual rows?

- A. GROUP BY
- B. HAVING
- C. WHERE
- D. ORDER BY

Answer: C

Q2

Which removes duplicate rows?

- A. UNIQUE
- B. DISTINCT
- C. REMOVE
- D. GROUP

Answer: B

Q3

How do you check NULL?

A.

WHERE salary = NULL

B.

WHERE salary IS NULL

C.

WHERE salary == NULL

Answer: B

Q4

Which function calculates average?

- A. SUM
- B. COUNT
- C. AVG
- D. MEAN

Answer: C

Q5

Which clause filters groups?

- A. WHERE
- B. HAVING
- C. FILTER
- D. GROUP

Answer: B

Intermediate Quiz

Q6

What is the difference between:

UNION

and

UNION ALL

Answer: UNION removes duplicates; UNION ALL retains duplicates.

Q7

Which JOIN keeps every row from the left table?

Answer: LEFT JOIN

Q8

What does EXISTS check?

Answer: Whether the subquery returns at least one row.

Q9

What is the purpose of **PARTITION BY**?

Answer: It divides rows into logical groups for window-function calculations.

Q10

Which function can assign sequential numbers?

Answer:

ROW_NUMBER()

ADVANCED QUIZ

Q11

Find the second-highest salary without using LIMIT/TOP.

Q12

Find employees earning more than their department average.

Q13

Find the highest-priced product in every category.

Q14

Find customers who placed orders in both 2025 and 2026.

Q15

Find the top three products by revenue within each category.

Q16

Find customers whose total spending is greater than the average customer spending.

Q17

Find products that have never been ordered.

Q18

Find the month with the highest revenue.

Q19

Find each customer's first order.

Q20

Find each customer's previous order amount.

CHALLENGES

Challenge 1 — Beginner

Find all products:

price > 10,000

Then sort from highest to lowest.

Challenge 2

Find products whose price is above the overall average price.

Challenge 3

Find the average product price for each category.

Challenge 4

Find categories having more than 10 products.

Challenge 5

Find customers who have never placed an order.

Challenge 6

Find customers whose total spending exceeds ₹1,00,000.

Challenge 7

Find the highest-selling product.

Challenge 8

Find the second-highest-selling product.

Challenge 9

Find the top three products in every category.

Challenge 10

Find the highest-spending customer in every state.

Challenge 11

Find monthly revenue.

Challenge 12

Find monthly revenue and previous month's revenue.

Use:

LAG()

Challenge 13

Calculate month-over-month growth.

Concept:

Current Month Revenue

-

Previous Month Revenue

Previous Month Revenue

Challenge 14

Find customers whose spending is greater than the average customer spending.

Challenge 15

Find products priced above their category's average price.

ASSIGNMENTS

Assignment 1 — SELECT Basics

Write queries to:

1. Display all customers.
 2. Display customer names only.
 3. Display unique cities.
 4. Display all products.
 5. Display product name and price.
 6. Create aliases.
 7. Calculate product value.
 8. Sort products by price.
 9. Display the 10 most expensive products.
 10. Display the 10 cheapest products.
-

Assignment 2 — Filtering

Write queries to:

1. Find products above ₹5,000.
 2. Find products below ₹1,000.
 3. Find products between ₹5,000 and ₹20,000.
 4. Find products belonging to Electronics.
 5. Find products belonging to Electronics or Furniture.
 6. Find customers from Tamil Nadu.
 7. Find customers whose names start with A.
 8. Find customers whose names contain **an**.
 9. Find products with NULL stock.
 10. Find products with non-NULL stock.
-

Assignment 3 — Aggregation

Write queries to calculate:

1. Total products.
2. Total customers.
3. Total orders.
4. Total revenue.
5. Average product price.
6. Minimum product price.

7. Maximum product price.
 8. Number of products per category.
 9. Revenue per customer.
 10. Revenue per category.
-

Assignment 4 — JOIN

Write queries to:

1. Display customers and their orders.
 2. Display orders and order items.
 3. Display products sold in orders.
 4. Display customer + order + product.
 5. Find customers without orders.
 6. Find products never sold.
 7. Find total customer spending.
 8. Find total product sales.
 9. Find category revenue.
 10. Find customer revenue by state.
-

Assignment 5 — Subqueries

Write queries to:

1. Products above average price.
 2. Products below average price.
 3. Employees above average salary.
 4. Employees above department average salary.
 5. Customers above average spending.
 6. Highest-priced product.
 7. Second-highest-priced product.
 8. Third-highest-priced product.
 9. Customers with orders.
 10. Customers without orders.
-

Assignment 6 — CASE

Create:

Budget
Mid Range
Premium
Luxury

based on product price.

Then create:

Low Value
Regular
Premium
VIP

based on customer spending.

Assignment 7 — CTE

Create queries for:

1. Customer revenue.
 2. Product revenue.
 3. Category revenue.
 4. Monthly revenue.
 5. Top customers.
 6. Top products.
 7. Customer segmentation.
 8. Monthly growth.
 9. Running revenue.
 10. Top products per category.
-

Assignment 8 — Window Functions

Write queries using:

ROW_NUMBER
RANK
DENSE_RANK
LAG
LEAD
SUM OVER
AVG OVER
PARTITION BY

Problems:

1. Rank employees by salary.
 2. Rank employees within department.
 3. Top 3 employees per department.
 4. First order per customer.
 5. Last order per customer.
 6. Previous order amount.
 7. Next order amount.
 8. Running revenue.
 9. Running customer spending.
 10. Seven-day moving average.
-

CAPSTONE PROJECT

ABC Retail SQL Analytics Platform

The final student must build a complete analytical SQL solution containing:

Module 1 — Customer Analytics

Total customers
Active customers
Inactive customers
Top customers
Customers without orders
Customer segmentation
Customer lifetime value

Module 2 — Product Analytics

Total products
Average price

Products above average
Most expensive products
Cheapest products
Top-selling products
Products never sold
Category comparison

Module 3 — Sales Analytics

Total sales
Daily sales
Monthly sales
Yearly sales
Average order value
Top orders
Revenue by category
Revenue by product
Revenue by customer

Module 4 — Advanced Analytics

Top 3 products per category
Top 3 customers per state
Customer ranking
Product ranking
Running revenue
Moving average
Month-over-month growth
First purchase
Last purchase
Previous purchase
Customer segmentation

Module 5 — Executive Dashboard Queries

The final application should provide approximately:

ABC RETAIL
SALES ANALYTICS

Total Customers	25,000
Total Orders	85,000
Total Products	5,000
Total Revenue	₹125 Cr

TOP PRODUCTS

Product	Revenue
Laptop	₹12 Cr
Mobile	₹10 Cr
TV	₹8 Cr

TOP CUSTOMERS

Customer	Revenue
Customer A	₹25 L
Customer B	₹22 L
Customer C	₹19 L

CATEGORY PERFORMANCE

Category	Revenue
Electronics	₹50 Cr
Furniture	₹25 Cr
Clothing	₹20 Cr

The important teaching point is that **every dashboard metric should be generated from **SELECT** queries.**

Recommended 30-Day SELECT-Only Training Plan

Day	Topic
1	SQL & SELECT fundamentals
2	SELECT columns & aliases
3	Expressions & calculations

- 4 DISTINCT
- 5 WHERE
- 6 AND / OR / NOT
- 7 IN / BETWEEN
- 8 LIKE
- 9 NULL
- 10 ORDER BY / LIMIT
- 11 String functions
- 12 Numeric functions
- 13 Date functions
- 14 COUNT / SUM / AVG
- 15 GROUP BY
- 16 HAVING
- 17 CASE
- 18 COALESCE / NULL handling
- 19 INNER JOIN
- 20 LEFT / RIGHT / FULL JOIN
- 21 Multiple JOINS
- 22 SELF JOIN / CROSS JOIN
- 23 Subqueries
- 24 Correlated subqueries
- 25 EXISTS / NOT EXISTS
- 26 UNION / INTERSECT / EXCEPT
- 27 CTE
- 28 Window functions

The Most Important SQL SELECT Patterns to Master

By the end of this course, students should be comfortable with these **12 patterns**:

```
-- 1. Basic SELECT
SELECT columns
FROM table;
-- 2. Filtering
SELECT columns
FROM table
WHERE condition;
-- 3. Aggregation
SELECT category, SUM(amount)
FROM sales
GROUP BY category;
-- 4. Group filtering
SELECT category, SUM(amount)
FROM sales
GROUP BY category
HAVING SUM(amount) > 100000;
-- 5. JOIN
SELECT ...
FROM A
JOIN B
ON A.id = B.id;
-- 6. Subquery
SELECT *
FROM products
WHERE price >
(
    SELECT AVG(price)
    FROM products
);
-- 7. EXISTS
SELECT *
```

```
FROM customers c
WHERE EXISTS
(
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

```
-- 8. CTE
WITH x AS
```

```
(
    SELECT ...
)
```

```
SELECT *
FROM x;
```

```
-- 9. CASE
```

```
SELECT
    CASE
        WHEN condition THEN result
        ELSE result
    END
```

```
FROM table;
```

```
-- 10. Window function
```

```
SELECT
    *,
    RANK() OVER(
        ORDER BY salary DESC
    )
```

```
FROM employees;
```

```
-- 11. Partitioned window
```

```
SELECT
    *,
    ROW_NUMBER() OVER(
        PARTITION BY department
        ORDER BY salary DESC
    )
```

```
FROM employees;
```

```
-- 12. Advanced analytical SELECT
```

```
WITH x AS
```

```
(
    SELECT ...
),
```

```
y AS
```

```
(
    SELECT ...
```

```
)  
SELECT  
    ...,  
    RANK() OVER(...) AS ranking,  
    LAG(...) OVER(...) AS previous_value  
FROM x  
JOIN y  
    ON ...;
```

Final Learning Outcome

The student should move from:

"I know SELECT."

to:

"I can answer a real business question using a single SELECT statement, or a combination of JOINS, aggregations, subqueries, CTEs, CASE expressions, set operators and window functions."

That is the right scope if you want to make **SELECT itself a complete SQL course**, rather than treating SELECT as only one chapter of SQL.